

Edge Artificial Intelligence for Privacy-Preserving Smart Home Surveillance: A Systems-Oriented Review

Academic-style research manuscript

Abstract

Smart home surveillance increasingly relies on computer vision to detect people, vehicles, and anomalous events. Cloud-centered processing can provide substantial computing capacity, but continuous transmission of household video creates latency, bandwidth, privacy, and availability concerns. This paper examines edge artificial intelligence as an alternative systems architecture in which inference is performed on or near the camera. We synthesize design principles for model selection, event filtering, data retention, security, and lifecycle management, and we propose a practical reference pipeline for privacy-preserving residential surveillance. The analysis indicates that the most effective edge systems do not merely move a cloud model onto a smaller device; instead, they co-design sensing, inference, storage, and human notification around explicit privacy and latency requirements. Key open challenges include model drift, adversarial robustness, heterogeneous accelerators, reliable multi-camera coordination, and transparent user control.

Keywords: edge AI; computer vision; smart surveillance; privacy; object detection; edge computing

1. Introduction

Computer vision has transformed video surveillance from passive recording into an event-driven sensing system. Modern object detectors can identify people, vehicles, animals, packages, and other categories in near real time, while trackers can maintain identities across consecutive frames. In a smart home, these capabilities can reduce the burden of reviewing continuous recordings by converting video streams into a smaller set of meaningful events.

A conventional architecture sends video to a remote server or cloud platform for inference and storage. This design simplifies centralized management and can provide powerful hardware, but it also exposes a structural tension: the most sensitive data in a home are generated continuously, while only a small fraction of frames are usually relevant to security decisions. Edge AI addresses this mismatch by performing inference close to the sensor and transmitting only selected metadata, alerts, or short evidence clips.

This paper presents a systems-oriented review of edge AI for residential surveillance. Rather than treating detection accuracy as the only objective, we consider latency, privacy, energy use, network dependence, maintainability, and human usability. The central argument is that an effective home-surveillance system should minimize unnecessary data movement while preserving enough evidence for users to understand and verify automated alerts.

2. Background and Architectural Context

Deep neural networks have substantially improved object detection. One-stage detectors such as the YOLO family emphasize fast inference, while two-stage approaches historically prioritized proposal quality and classification accuracy. For surveillance, the model is only one component of a larger pipeline that includes video decoding, frame sampling, preprocessing, detection, tracking, event logic, persistence, and notification.

Edge computing places some or all of these functions near the data source. A camera may contain an embedded neural processor, or a local gateway may aggregate streams from several cameras. The gateway model is particularly attractive for homes because it can provide stronger compute than an individual camera while keeping raw video inside the local network. Cloud services can remain useful for optional backup, remote access, fleet management, or computationally expensive secondary analysis.

Privacy preservation at the edge should be understood as data minimization rather than absolute secrecy. Local inference reduces the need to upload raw streams, but the device still handles sensitive information and therefore requires secure storage, authentication, encryption, software updates, and carefully designed retention policies.

3. Reference Pipeline for an Edge Surveillance System

A practical pipeline begins with configurable video sources such as RTSP cameras and local MP4 files used for testing. Frames are decoded and sampled according to the available compute budget. An object detector identifies relevant classes, and a tracker associates detections across time. Event rules then convert low-level observations into higher-level events, for example a person entering a restricted zone, a vehicle remaining near an entrance, or repeated motion during a configured period.

The event layer is important because storing every positive detection can create redundant records. Temporal aggregation can merge adjacent detections into a single event with a start time, end time, representative confidence score, and evidence snapshot. Short video clips may be retained only for events that exceed a risk threshold or that users explicitly mark for preservation.

Metadata and media should be separated logically. A relational database can store event identifiers, timestamps, camera identifiers, classes, confidence values, zones, and review status, while images or clips are stored in an object store or filesystem. This avoids inflating database rows with large binary objects and makes retention policies easier to implement.

4. Design Trade-offs

Edge surveillance requires balancing competing objectives. Smaller models reduce latency and power consumption but may lose accuracy on small or partially occluded objects. Higher input resolution can improve detection of distant subjects but increases decoding, memory, and inference cost. Frame skipping reduces compute but can miss short events. Tracking can compensate for sparse inference, although tracking itself introduces failure modes during occlusion or abrupt camera motion.

Privacy and forensic value also pull in different directions. A system that stores no video maximizes data minimization but may provide insufficient evidence after an incident. Conversely, continuous recording offers strong retrospective evidence but increases privacy exposure and storage requirements. Event-triggered retention is a useful compromise when combined with clear user controls.

Network resilience is another advantage of local processing. A home should continue to detect and record critical events during an internet outage. Remote notifications may be delayed, but local inference and storage can remain operational. This suggests that cloud connectivity should be an enhancement rather than a hard dependency for core safety functions.

Design dimension	Edge-first choice	Cloud-first choice	Primary trade-off
Inference	Local gateway/device	Remote GPU service	Latency vs. compute scale
Video retention	Event-triggered local clips	Continuous remote archive	Privacy vs. forensic depth
Connectivity	Core functions offline	Internet-dependent	Resilience vs. simplicity
Model size	Compact/quantized	Larger model	Efficiency vs. accuracy

Table 1. Representative system-level trade-offs in edge and cloud surveillance architectures.

5. Security and Privacy Engineering

The security boundary of an edge system includes cameras, the local gateway, storage, mobile or web clients, and any optional cloud endpoint. Device credentials should not be embedded in source code, and administrative interfaces should use authenticated sessions with least-privilege access. Data at rest should be encrypted when feasible, while network traffic should use secure protocols and isolated local-network segments.

Privacy controls should be visible to users. Useful mechanisms include configurable retention periods, privacy masks for sensitive regions, per-camera recording modes, event deletion, export controls, and an audit trail for administrative actions. Where face recognition is not required, a system can deliberately avoid identity inference and focus on non-biometric categories such as person, vehicle, and package.

Software supply-chain security is equally important. Models and application packages should be obtained from trusted sources, verified, and updated through a controlled mechanism. Because edge devices may remain deployed for years, maintainability and patching are first-class system requirements rather than post-deployment conveniences.

6. Evaluation Methodology

Evaluation should combine model metrics with end-to-end system metrics. Detection precision and recall remain important, but a household user experiences alert quality rather than frame-level accuracy. Event-level precision, missed-event rate, duplicate-alert rate, notification latency, storage growth, power draw, and recovery after network failure provide a more realistic assessment.

Testing should include daylight and low-light conditions, occlusion, multiple subjects, camera noise, varying distances, and ordinary household activities that may cause false alarms. Local MP4 files are useful for reproducible regression testing, while live RTSP streams are necessary for evaluating timing, buffering, and connection recovery.

A strong benchmark should also report the hardware platform, model version, input resolution, inference backend, quantization mode, frame-sampling strategy, and confidence thresholds. Without these details, throughput numbers are difficult to reproduce or compare.

7. Open Challenges and Future Research

Model drift is a major challenge because a home environment changes over time. Furniture moves, lighting changes, cameras are repositioned, and seasonal conditions

alter the visual distribution. Lightweight mechanisms for detecting performance degradation without uploading private video remain an important research direction.

Another challenge is trustworthy alert reasoning. A user should be able to understand why an event was created, which frames supported it, and how confidence evolved. This favors systems that preserve structured event provenance rather than exposing only a final label.

Future systems may combine vision with door sensors, microphones, radar, or smart locks. Multimodal fusion can reduce ambiguity, but it also expands the privacy surface. Research should therefore evaluate not only whether additional sensors improve accuracy, but whether the incremental information is proportionate to the security benefit.

8. Conclusion

Edge AI provides a compelling architecture for smart home surveillance because it aligns computation with the location where sensitive data are produced. Its principal value is not simply lower latency; it enables a data-minimizing design in which raw video can remain local and only meaningful events need to be retained or transmitted.

Successful deployments require co-design across machine learning, systems engineering, security, and user experience. An accurate detector cannot compensate for poor retention policies, insecure credentials, unreliable event logic, or excessive false alerts. Future progress will depend on efficient models, transparent event reasoning, robust lifecycle management, and privacy controls that remain understandable to non-expert users.

References

- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of CVPR*, 779-788.
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *Advances in Neural Information Processing Systems*, 28.
- Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal*, 3(5), 637-646.
- Satyanarayanan, M. (2017). The Emergence of Edge Computing. *Computer*, 50(1), 30-39.
- European Parliament and Council of the European Union. (2016). Regulation (EU) 2016/679 (General Data Protection Regulation).
- NIST. (2020). NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management, Version 1.0.
- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal Speed and Accuracy of Object Detection. *arXiv:2004.10934*.
- Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple Online and Realtime Tracking. *IEEE International Conference on Image Processing*, 3464-3468.